



WANT TO SKIP THE DOCS?



"If you're serious about *really* understanding React Query, there's no better way than with query.gg"
—Tanner Linsley

To help make query.gg more accessible, you can enable a regional discount of 60% off for being in  India.

GET 60% OFF 

Guides & Concepts

On this page ▾

Advanced Server Rendering

 Copy page ▾

Welcome to the Advanced Server Rendering guide, where you will learn all about using React Query with streaming, Server Components and the Next.js app router.

You might want to read the [Server Rendering & Hydration guide](#) before this one as it teaches the basics for using React Query with SSR and



well as [Prefetching & Router Integration](#) also contains valuable background.

Before we start, let's note that while the `initialData` approach outlined in the SSR guide also works with Server Components, we'll focus this guide on the hydration APIs.

Server Components & Next.js app router

We won't cover Server Components in depth here, but the short version is that they are components that are guaranteed to *only* run on the server, both for the initial page view and **also on page transitions**. This is similar to how Next.js `getServerSideProps` / `getStaticProps` and Remix `loader` works, as these also always run on the server but while those can only return data, Server Components can do a lot more. The data part is central to React Query however, so let's focus on that.

How do we take what we learned in the Server Rendering guide about [passing data prefetched in framework loaders to the app](#) and apply that to Server Components and the Next.js app router? The best way to start thinking about this is to consider Server



framework loader.

A quick note on terminology

So far in these guides, we've been talking about the *server* and the *client*. It's important to note that confusingly enough this does not match 1-1 with *Server Components* and *Client Components*. Server Components are guaranteed to only run on the server, but Client Components can actually run in both places. The reason for this is that they can also render during the initial *server rendering* pass.

One way to think of this is that even though Server Components also *render*, they happen during a "loader phase" (always happens on the server), while Client Components run during the "application phase". That application can run both on the server during SSR, and in for example a browser. Where exactly that application runs and if it runs during SSR or not might differ between frameworks.

Initial setup

The first step of any React Query setup is always to create a `queryClient` and wrap your application in a

`QueryClientProvider` With Server



same across frameworks, one difference being the filename conventions:

```
tsx
// In Next.js, this file would be called:
'use client'

// Since QueryClientProvider relies on useQueryClient, we need to use 'use client'
import {
  isServer,
  QueryClient,
  QueryClientProvider,
} from '@tanstack/react-query'

function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        // With SSR, we usually want to set default staleTime to 0
        // above 0 to avoid refetching in SSR
        staleTime: 60 * 1000,
      },
    },
  })
}

let browserQueryClient: QueryClient | undefined = undefined

function getQueryClient() {
  if (isServer) {
    // Server: always make a new query client
    return makeQueryClient()
  } else {
    // Browser: make a new query client if not already present
    // This is very important, so we don't
    // suspend during the initial render
    // have a suspense boundary BELOW the
    if (!browserQueryClient) browserQueryClient = makeQueryClient()
    return browserQueryClient
  }
}

export default function Providers({ children }) {
  // NOTE: Avoid useState when initializing
  // query client here.
  // See https://tanstack.com/query/v5/docs/framework/react/guides/advanced-ssr
```



```
const queryClient = getQueryClient()

return (
  <QueryClientProvider client={queryClient}>
  )
}
```

tsx

```
// In Next.js, this file would be called:
import Providers from './providers'

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <head />
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  )
}
```

This part is pretty similar to what we did in the SSR guide, we just need to split things up into two different files.

Prefetching and de/hydrating data

Next, let's look at how to actually prefetch data, then dehydrate and hydrate it. This is what it looked like using the **Next.js Pages Router**:

tsx

```
// pages/posts.tsx
import {
```



```
    QueryClient,
    useQuery,
  } from '@tanstack/react-query'

  // This could also be getServerSideProps
  export async function getStaticProps() {
    const queryClient = new QueryClient()

    await queryClient.prefetchQuery({
      queryKey: ['posts'],
      queryFn: getPosts,
    })

    return {
      props: {
        dehydratedState: dehydrate(queryClient),
      },
    }
  }

  function Posts() {
    // This useQuery could just as well happen on the server
    // the <PostsRoute>, data will be available
    //
    // Note that we are using useQuery here instead of useSuspenseQuery
    // Because this data has already been prefetched, it will not
    // ever suspend in the component itself
    // prefetch, this will instead fetch the data on the client
    // using useSuspenseQuery would have had the same effect
    const { data } = useQuery({ queryKey: ['posts'], queryFn: getPosts })

    // This query was not prefetched on the server, so it will
    // fetch until on the client, both for SSR and for the browser
    const { data: commentsData } = useQuery({
      queryKey: ['posts-comments'],
      queryFn: getComments,
    })

    // ...

  }

  export default function PostsRoute({ dehydratedState }) {
    return (
      <HydrationBoundary state={dehydratedState}>
        <Posts />
      </HydrationBoundary>
    )
  }
}
```



Converting this to the app router actually looks pretty similar, we just need to move things around a bit. First, we'll create a Server Component to do the prefetching part:

```
tsx
// app/posts/page.tsx
import {
  dehydrate,
  HydrationBoundary,
  QueryClient,
} from '@tanstack/react-query'
import Posts from './posts'

export default async function PostsPage() {
  const queryClient = new QueryClient()

  await queryClient.prefetchQuery({
    queryKey: ['posts'],
    queryFn: getPosts,
  })

  return (
    // Neat! Serialization is now as easy
    // HydrationBoundary is a Client Comp
    <HydrationBoundary state={dehydrate(c
      <Posts />
    </HydrationBoundary>
  )
}
```

Next, we'll look at what the Client Component part looks like:

```
tsx
// app/posts/posts.tsx
'use client'

export default function Posts() {
```



```
const { data } = useQuery({
  queryKey: ['posts'],
  queryFn: () => getPosts(),
})

// This query was not prefetched on the
// fetching until on the client, both
const { data: commentsData } = useQuery({
  queryKey: ['posts-comments'],
  queryFn: getComments,
})

// ...
}
```

One neat thing about the examples above is that the only thing that is Next.js-specific here are the file names, everything else would look the same in any other framework that supports Server Components.

In the SSR guide, we noted that you could get rid of the boilerplate of having `<HydrationBoundary>` in every route.

This is not possible with Server Components.

“NOTE: If you encounter a type error while using async Server Components with TypeScript versions lower than 5.1.3 and @types/react versions lower than 18.2.8, it is recommended to update to the latest versions of both. Alternatively, you can use the temporary workaround of adding //



*Component */} when calling this component inside another. For more information, see [Async Server Component TypeScript Error in the Next.js 13 docs](#)."*

"NOTE: If you encounter an error [Only plain objects, and a few built-ins, can be passed to Server Actions](#). Classes or null prototypes are not supported. make sure that you're not passing to queryFn a function reference, instead call the function because queryFn args has a bunch of properties and not all of it would be serializable. see [Server Action only works when queryFn isn't a reference](#)."

Nesting Server Components

A nice thing about Server Components is that they can be nested and exist on many levels in the React tree, making it possible to prefetch data closer to where it's actually used instead of only at the top of the application (just like Remix loaders). This can be as simple as a Server Component rendering another Server Component (we'll leave the Client Components out in this example for brevity):

```
tsx
```



```
    dehydrate,  
    HydrationBoundary,  
    QueryClient,  
  } from '@tanstack/react-query'  
  import Posts from './posts'  
  import CommentsServerComponent from './comments-server'  
  
  export default async function PostsPage() {  
    const queryClient = new QueryClient()  
  
    await queryClient.prefetchQuery({  
      queryKey: ['posts'],  
      queryFn: getPosts,  
    })  
  
    return (  
      <HydrationBoundary state={dehydrate(queryClient)}>  
        <Posts />  
        <CommentsServerComponent />  
      </HydrationBoundary>  
    )  
  }  
  
  // app/posts/comments-server.tsx  
  import {  
    dehydrate,  
    HydrationBoundary,  
    QueryClient,  
  } from '@tanstack/react-query'  
  import Comments from './comments'  
  
  export default async function CommentsServerPage() {  
    const queryClient = new QueryClient()  
  
    await queryClient.prefetchQuery({  
      queryKey: ['posts-comments'],  
      queryFn: getComments,  
    })  
  
    return (  
      <HydrationBoundary state={dehydrate(queryClient)}>  
        <Comments />  
      </HydrationBoundary>  
    )  
  }  
}
```



`<HydrationBoundary>` in multiple places, and create and dehydrate multiple `queryClient` for prefetching.

Note that because we are awaiting `getPosts` before rendering `CommentsServerComponent` this would lead to a server side waterfall:

```
1. ▶ getPosts()
2.   ▶ getComments()
```

If the server latency to the data is low, this might not be a huge issue, but is still worth pointing out.

In Next.js, besides prefetching data in `page.tsx`, you can also do it in `layout.tsx`, and in parallel routes.

Because these are all part of the routing, Next.js knows how to fetch them all in parallel. So if `CommentsServerComponent` above was instead expressed as a parallel route, the waterfall would be flattened automatically.

As more frameworks start supporting Server Components, they might have other routing conventions. Read your framework docs for details.

Alternative: Use a single



In the example above, we create a new `queryClient` for each Server Component that fetches data. This is the recommended approach, but if you want to, you can alternatively create a single one that is reused across all Server Components:

```
tsx
// app/getQueryClient.tsx
import { QueryClient } from '@tanstack/react-query'
import { cache } from 'react'

// cache() is scoped per request, so we can use it to create a single client
const getQueryClient = cache(() => new QueryClient())
export default getQueryClient
```

The benefit of this is that you can call `getQueryClient()` to get a hold of this client anywhere that gets called from a Server Component, including utility functions. The downside is that every time you call `dehydrate(getQueryClient())`, you serialize *the entire* `queryClient`, including queries that have already been serialized before and are unrelated to the current Server Component which is unnecessary overhead.

Next.js already dedupes requests that utilize `fetch()`, but if you are using something else in your `queryFn`, or if you use a framework that does *not*



described above might make sense, despite the duplicated serialization.

"As a future improvement, we might look into creating a `dehydrateNew()` function (name pending) that only dehydrate queries that are new since the last call to `dehydrateNew()`. Feel free to get in touch if this sounds interesting and like something you want to help out with!"

Data ownership and revalidation

With Server Components, it's important to think about data ownership and revalidation. To explain why, let's look at a modified example from above:

```
tsx
// app/posts/page.tsx
import {
  dehydrate,
  HydrationBoundary,
  QueryClient,
} from '@tanstack/react-query'
import Posts from './posts'

export default async function PostsPage() {
  const queryClient = new QueryClient()

  // Note we are now using fetchQuery()
  const posts = await queryClient.fetchQuery({
    queryKey: ['posts'],
    queryFn: getPosts,
  })
}
```



```
<HydrationBoundary state={queryState}>
  { /* This is the new part */ }
  <div>Nr of posts: {posts.length}</div>
  <Posts />
</HydrationBoundary>
)
```

We are now rendering data from the `getPosts` query both in a Server Component and in a Client Component. This will be fine for the initial page render, but what happens when the query revalidates on the client for some reason when `staleTime` has been passed?

React Query has no idea of how to *revalidate the Server Component*, so if it refetches the data on the client, causing React to rerender the list of posts, the `Nr of posts: {posts.length}` will end up out of sync.

This is fine if you set `staleTime: Infinity`, so that React Query never revalidates, but this is probably not what you want if you are using React Query in the first place.

Using React Query with Server Components makes most sense if:

- You have an app using React Query and want to migrate to Server



- You want a familiar programming paradigm, but want to still sprinkle in the benefits of Server Components where it makes most sense
- You have some use case that React Query covers, but that your framework of choice does not cover

It's hard to give general advice on when it makes sense to pair React Query with Server Components and not. **If you are just starting out with a new Server Components app, we suggest you start out with any tools for data fetching your framework provides you with and avoid bringing in React Query until you actually need it.** This might be never, and that's fine, use the right tool for the job!

If you do use it, a good rule of thumb is to avoid `queryClient.fetchQuery` unless you need to catch errors. If you do use it, don't render its result on the server or pass the result to another component, even a Client Component one.

From the React Query perspective, treat Server Components as a place to prefetch data, nothing more.



Components own some data, and Client Components own other, just make sure those two realities don't get out of sync.

Streaming with Server Components

The Next.js app router automatically streams any part of the application that is ready to be displayed to the browser as soon as possible, so finished content can be displayed immediately without waiting for still pending content. It does this along `<Suspense>` boundary lines. Note that if you create a file `loading.tsx`, this automatically creates a `<Suspense>` boundary behind the scenes.

With the prefetching patterns described above, React Query is perfectly compatible with this form of streaming. As the data for each Suspense boundary resolves, Next.js can render and stream the finished content to the browser. This works even if you are using `useQuery` as outlined above because the suspending actually happens when you `await` the prefetch.

As of React Query v5.40.0, you don't



dehydrated and sent to the client. This lets you kick off prefetches as early as possible without letting them block an entire Suspense boundary, and streams the *data* to the client as the query finishes. This can be useful for example if you want to prefetch some content that is only visible after some user interaction, or say if you want to `await` and render the first page of an infinite query, but start prefetching page 2 without blocking rendering.

To make this work, we have to instruct the `queryClient` to also `dehydrate` pending Queries. We can do this globally, or by passing that option directly to `dehydrate`.

We will also need to move the `getQueryClient()` function out of our `app/providers.tsx` file as we want to use it in our server component and our client provider.

tsx

```
// app/get-query-client.ts
import {
  isServer,
  QueryClient,
  defaultShouldDehydrateQuery,
} from '@tanstack/react-query'

function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
```



```

    // include pending queries in default
    shouldDehydrateQuery: (query) =>
      defaultShouldDehydrateQuery(query),
    query.state.status === 'pending',
    shouldRedactErrors: (error) => {
      // We should not catch Next.js
      // as that's how Next.js detects
      // so we cannot redact them.
      // Next.js also automatically
      // with better digests.
      return false
    },
  },
},
},
})
}

let browserQueryClient: QueryClient | undefined

export function getQueryClient() {
  if (isServer) {
    // Server: always make a new query client
    return makeQueryClient()
  } else {
    // Browser: make a new query client
    // This is very important, so we don't
    // suspend during the initial render
    // have a suspense boundary BELOW the
    if (!browserQueryClient) browserQueryClient =
      makeQueryClient()
    return browserQueryClient
  }
}

```

“Note: This works in NextJs and Server Components because React can serialize Promises over the wire when you pass them down to Client Components.”

Then, all we need to do is provide a

`HydrationBoundary` but we don't need



```
tsx
// app/posts/page.tsx
import { dehydrate, HydrationBoundary } from '@tanstack/react-query'
import { getQueryClient } from './get-query-client'
import Posts from './posts'

// the function doesn't need to be `async`
export default function PostsPage() {
  const queryClient = getQueryClient()

  // look ma, no await
  queryClient.prefetchQuery({
    queryKey: ['posts'],
    queryFn: getPosts,
  })

  return (
    <HydrationBoundary state={dehydrate(queryClient)}>
      <Posts />
    </HydrationBoundary>
  )
}
```

On the client, the Promise will be put into the QueryCache for us. That means we can now call `useSuspenseQuery` inside the `Posts` component to "use" that Promise (which was created on the Server):

```
tsx
// app/posts/posts.tsx
'use client'

export default function Posts() {
  const { data } = useSuspenseQuery({ queryKey: ['posts'], queryFn: getPosts })

  // ...
}
```



the Promise would still be picked up correctly. However, NextJs won't suspend in that case and the component will render in the `pending` status, which also opts out of server rendering the content."

If you're using non-JSON data types and serialize the query results on the server, you can specify the `dehydrate.serializeData` and `hydrate.deserializeData` options to serialize and deserialize the data on each side of the boundary to ensure the data in the cache is the same format both on the server and the client:

```
tsx
// app/get-query-client.ts
import { QueryClient, defaultShouldDehydrateQuery } from '@tanstack/query-core'
import { deserialize, serialize } from 'serdex'

function makeQueryClient() {
  return new QueryClient({
    defaultOptions: {
      // ...
      hydrate: {
        deserializeData: deserialize,
      },
      dehydrate: {
        serializeData: serialize,
      },
    },
  })
}

// ...
```

```
tsx
// app/posts/page.tsx
```



```
HydrationBoundary,  
QueryClient,  
} from '@tanstack/react-query'  
import { getQueryClient } from './get-query-client'  
import { serialize } from './transformer'  
import Posts from './posts'  
  
export default function PostsPage() {  
  const queryClient = getQueryClient()  
  
  // look ma, no await  
  queryClient.prefetchQuery({  
    queryKey: ['posts'],  
    queryFn: () => getPosts().then(serialize)  
  })  
  
  return (  
    <HydrationBoundary state={dehydrate(queryClient)}>  
      <Posts />  
    </HydrationBoundary>  
  )  
}
```

tsx

```
// app/posts/posts.tsx  
'use client'  
  
export default function Posts() {  
  const { data } = useSuspenseQuery({ queryKey: ['posts'],  
    queryFn: () => getPosts().then(serialize)  
  })  
  
  // ...  
}
```

Now, your `getPosts` function can return e.g. `Temporal` datetime objects and the data will be serialized and deserialized on the client, assuming your transformer can serialize and deserialize those data types.

For more information, check out the [Next.js App with Prefetching Example](#).



Using the Persist Adapter with Streaming

If you're using the persist adapter with the [Streaming with Server Components](#) feature, you need to be careful not to save promises to storage. Since pending queries can be dehydrated and streamed to the client, you should configure the persister to only persist successful queries:

```
tsx
<PersistQueryClientProvider
  client={queryClient}
  persistOptions={{
    persister,
    // We don't want to save promises in
    dehydrateOptions: { shouldDehydrateQuery: (query) => !query.isPromise },
  }}
>
  {children}
</PersistQueryClientProvider>
```

This ensures that only successfully resolved queries are persisted to storage, preventing serialization issues with pending promises.

Experimental streaming without prefetching in Next.js

While we recommend the prefetching solution detailed above because it



page navigation, there is an experimental way to skip prefetching altogether and still have streaming SSR work: `@tanstack/react-query-next-experimental`

This package will allow you to fetch data on the server (in a Client Component) by just calling `useSuspenseQuery` in your component. Results will then be streamed from the server to the client as `SuspenseBoundaries` resolve. If you call `useSuspenseQuery` without wrapping it in a `<Suspense>` boundary, the HTML response won't start until the fetch resolves. This can be when you want depending on the situation, but keep in mind that this will hurt your TTFB.

To achieve this, wrap your app in the `ReactQueryStreamedHydration` component:

```
tsx
// app/providers.tsx
'use client'

import {
  isServer,
  QueryClient,
  QueryClientProvider,
} from '@tanstack/react-query'
import * as React from 'react'
import { ReactQueryStreamedHydration } from '@tanstack/react-query-next-experimental'
```



```
    queries: {
      // With SSR, we usually want to set a staleTime of 0
      // above 0 to avoid refetching in SSR
      staleTime: 60 * 1000,
    },
  },
})
}

let browserQueryClient: QueryClient | undefined = undefined

function getQueryClient() {
  if (isServer) {
    // Server: always make a new query client
    return makeQueryClient()
  } else {
    // Browser: make a new query client if we don't already have one
    // This is very important, so we don't
    // suspend during the initial render
    // have a suspense boundary BELOW the
    if (!browserQueryClient) browserQueryClient = makeQueryClient()
    return browserQueryClient
  }
}

export function Providers(props: { children: React.ReactNode }) {
  // NOTE: Avoid useState when initializing
  //        have a suspense boundary between
  //        the initialization and the
  //        suspend because React will throw
  //        the error and render if it suspends and then
  const queryClient = getQueryClient()

  return (
    <QueryClientProvider client={queryClient}>
      <ReactQueryStreamedHydration>
        {props.children}
      </ReactQueryStreamedHydration>
    </QueryClientProvider>
  )
}
```

For more information, check out the [NextJs Suspense Streaming Example](#).



need to prefetch queries manually to have SSR work, and it even still streams in the result! This gives you phenomenal DX and lower code complexity.

The downside is easiest to explain if we look back at [the complex request waterfall example](#) in the Performance & Request Waterfalls guide. Server Components with prefetching effectively eliminates the request waterfalls both for the initial page load **and** any subsequent navigation. This prefetch-less approach however will only flatten the waterfalls on the initial page load but ends up the same deep waterfall as the original example on page navigations:

1. ▷ JS for <Feed>
2. ▷ getFeed()
3. ▷ JS for <GraphFeedItem>
4. ▷ getGraphDataById()

This is even worse than with `getServerSideProps` / `getStaticProps`, since with those we could at least parallelize data- and code-fetching.

If you value DX/iteration/shipping speed with low code complexity over performance, don't have deeply nested queries, or are on top of your request



tools like `useSuspenseQueries`, this can be a good tradeoff.

"It might be possible to combine the two approaches, but even we haven't tried that out yet. If you do try this, please report back your findings, or even update these docs with some tips!"

Final words

Server Components and streaming are still fairly new concepts and we are still figuring out how React Query fits in and what improvements we can make to the API. We welcome suggestions, feedback and bug reports!

Similarly, it would be impossible to teach all the intricacies of this new paradigm all in one guide, on the first try. If you are missing some piece of information here or have suggestions on how to improve this content, also get

Further reading

To understand if your application can



using server components, see the article [You Might Not Need React Query](#).

[Edit on GitHub](#)